

1교시: Week2 10분 요약 + MSA를 운영 토폴로지로 보기

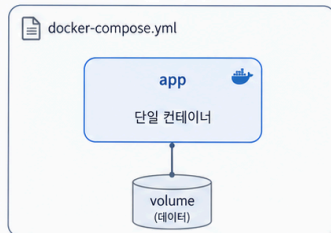
Week 3 Day 1 Lesson 1

2주차 복습에서 MSA 목표로 확장

Docker Compose로 실행하던 단일 애플리케이션을 여러 서비스 구조로 확장합니다.

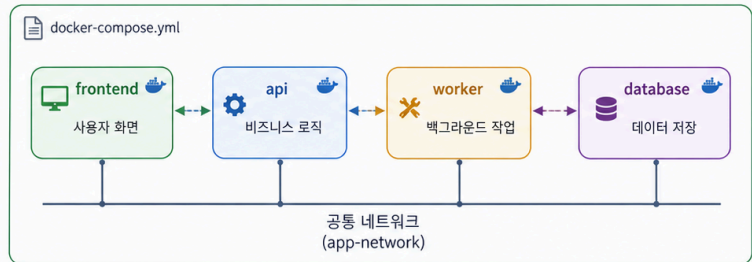
Docker 복습

단일 서비스



모든 기능이 하나의 컨테이너에 존재 (웹, API, 작업, 데이터)

여러 서비스 (MSA 지향 구조)



역할별 서비스 분리로 독립성, 확장성, 유지보수성 향상

실행 조건



포트

- frontend : 80
- api : 8080
- worker : - (내부 통신)
- database : 5432



로그

- 각 서비스 로그 분리
- docker compose logs -f
- 문제 추적 용이



health

- 각 서비스 healthcheck 설정
- 의존 서비스 상태 확인
- 자동 복구/재시작 기반



의존성 순서

- database
- ↓
- api
- ↓
- frontend / worker



MSA 목표

- ☑ 역할별 서비스 분리
- ☑ 독립적 배포와 확장
- ☑ 장애 격리와 안정성 향상
- ☑ 기술 스택의 유연성 확보



핵심 포인트: 단일 서비스에서 여러 서비스로의 전환은 "역할 분리"가 핵심입니다.

수업 목표

- Week2에서 배운 Docker image, container, port, env, logs, volume을 MSA service 단위로 다시 묶는다.
- 단일 container 정상과 전체 서비스 정상의 차이를 설명한다.
- MSA를 코드 구조가 아니라 운영 토폴로지, 의존성, 장애 전파의 문제로 읽는다.

왜 다시 정리하는가

Week2에서는 container 하나를 어떻게 실행하고 확인할지 배웠다. 하지만 실제 서비스는 대개 container 하나로 끝나지 않는다. 사용자는 frontend로 들어오고, frontend는 api를 호출하고, api는 database에 붙고, worker는 background에서 api나 queue를 확인한다.

따라서 Week3의 질문은 다음으로 바뀐다.

container 하나가 커졌는가?

에서

여러 service가 서로 기대한 주소, port, 설정, readiness 상태로 연결되어 있는가?

로 넘어간다.

Week2 개념을 MSA로 연결하기

Week2 개념	단일 container에서의 의미	MSA에서 다시 보는 의미
image	실행 파일 묶음	service별 배포 단위
container	실행 중인 process	service instance
port publish	host에서 접근할 통로	외부 진입점과 debug port 구분
environment	runtime config	service 간 주소, credential, feature flag
logs	process 출력	여러 service의 request 흐름 증거
volume	data 보존	stateful service의 lifecycle
network	container 통신	service boundary와 dependency map

MSA 수업에서는 명령을 더 많이 외우는 것이 목표가 아니다. 같은 명령을 여러 service에 적용하면서 어느 service가 전체 장애의 단서인지 찾아내는 것이 목표다.

표준 실습 앱 미리보기

Day1~Day2는 같은 앱을 쓴다.

```
cd week3/day1/labs/msa-demo
```

구조:

Service	역할	외부 접근	내부 주소	주요 증거
frontend	사용자 진입점, nginx reverse proxy	localhost:18083	frontend:80	browser, nginx log
api	상태 API, DB 연결 확인	localhost:18084	api:8080	/health, /api/status, api log
worker	background에서 API 상태 확인	없음	worker	worker log
db	PostgreSQL	없음	db:5432	healthcheck, db log, volume

요청 흐름:

```
browser -> frontend:80 -> api:8080 -> db:5432
worker  -> api:8080  -> db:5432
```

여기서 중요한 점은 worker가 사용자 요청을 직접 받지 않는다는 것이다. 하지만 API나 DB 장애가 나면 worker도 실패 로그를 남긴다. 이처럼 MSA에서는 사용자 경로와 background 경로를 구분해야 한

다.

첫 확인 명령

아직 실행하지 않고 파일을 먼저 읽는다.

```
cd week3/day1/labs/msa-demo
docker compose config
```

docker compose config에서 확인할 것:

확인 지점	왜 보는가
frontend.ports	사용자가 어디로 들어오는지 확인
api.ports와 api.expose	host debug port와 internal port 구분
api.environment.DB_HOST	API가 DB를 어떤 service name으로 찾는지 확인
worker.environment.API_URL	worker가 API를 어떤 주소로 호출하는지 확인
db.healthcheck	DB 준비 상태를 무엇으로 판단하는지 확인
volumes.msa-db-data	DB data가 container 삭제 후에도 남을 수 있는지 확인

강의 포인트

단일 container 수업에서는 docker ps에 Up이 뜨면 꽤 많은 것이 해결됐다. MSA에서는 다르다.

```
frontend Up
api Up
db Up
worker Up
```

이 상태여도 전체 서비스가 정상이라는 보장은 없다.

예를 들어 다음 상황은 모두 다르다.

상황	container 상태	사용자 경험	먼저 볼 증거
frontend만 정상	frontend Up	화면은 뜨지만 데이터 없음	nginx log, api URL
api는 Up, DB 연결 실패	api Up	API 503 또는 JSON error	/health, api log
db는 늦게 준비됨	db starting	API readiness 실패	db healthcheck
worker 실패	worker Up 또는 restart	사용자 화면은 정상일 수 있음	worker log

Evidence Note

W3D1S1 Evidence

- 내가 찾은 외부 진입점:
- 내부 service name:
- stateful service:
- 단일 container 정상과 MSA 정상의 차이:

다음 교시 연결

다음 교시에서는 Monolith와 MSA를 비교한다. 핵심은 MSA가 더 현대적이라는 이야기가 아니라, 배포 단위와 장애 영향 범위가 달라진다는 점이다.